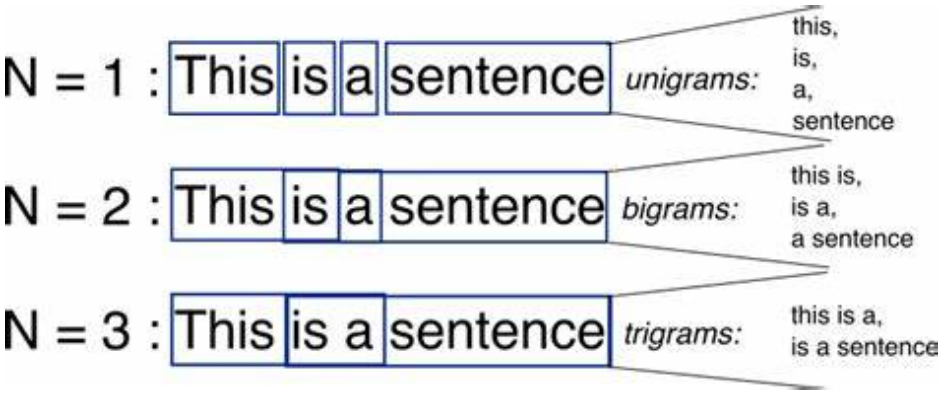
Mohaiya aldossary
Student ID: 2230004579

N-Grams

It's a probabilistic model that's trained on a corpus of text. Such a model is useful in many NLP applications including speech recognition, machine translation and predictive text input. An N-gram model is built by counting how often word sequences occur in corpus text and then estimating the probabilities.

Types of N-Grams



Use-cases of N-Grams

- Auto completion of sentences
- Auto spell check
- Voice-based personal assistant bots

Unigram

Unigram is an n-gram consisting of a single item of a sequence.

- It is commonly used to calculate the probability of finding a word in an article.

$$P(A) = \frac{\text{count of } A}{\text{count of all words}}$$

In [1]:

```
%pip install nltk pandas emoji scikit-learn

import nltk
from pathlib import Path
nltk_data_dir = Path("nltk_data").resolve()
nltk_data_dir.mkdir(exist_ok=True)
nltk.data.path = [str(nltk_data_dir)]
nltk.download("punkt_tab", download_dir=str(nltk_data_dir), quiet=True, raise_on_error=True)
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

unigrams = ngrams(word_tokenize(string), 1)

for item in unigrams:
    print(item)
```

```
Requirement already satisfied: nltk in c:\users\ezio9\appdata\local\programs\python\python311\lib\site-packages (3.9.2)
Requirement already satisfied: pandas in c:\users\ezio9\appdata\local\programs\python\python311\lib\site-packages (3.0.1)
Requirement already satisfied: emoji in c:\users\ezio9\appdata\local\programs\python\python311\lib\site-packages (2.15.0)
Requirement already satisfied: scikit-learn in c:\users\ezio9\appdata\local\programs\python\python311\lib\site-packages (1.8.0)
Requirement already satisfied: click in c:\users\ezio9\appdata\local\programs\python\python311\lib\site-packages (from nltk)
(8.3.1)
Requirement already satisfied: joblib in c:\users\ezio9\appdata\local\programs\python\python311\lib\site-packages (from nltk)
(1.5.3)
Requirement already satisfied: regex>=2021.8.3 in c:\users\ezio9\appdata\local\programs\python\python311\lib\site-packages (from
nltk) (2026.2.19)
Requirement already satisfied: tqdm in c:\users\ezio9\appdata\local\programs\python\python311\lib\site-packages (from nltk)
(4.67.3)
Requirement already satisfied: numpy>=1.26.0 in c:\users\ezio9\appdata\local\programs\python\python311\lib\site-packages (from
pandas) (2.4.2)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\ezio9\appdata\roaming\python\python311\site-packages (from
pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\users\ezio9\appdata\local\programs\python\python311\lib\site-packages (from pandas)
(2025.3)
Requirement already satisfied: scipy>=1.10.0 in c:\users\ezio9\appdata\local\programs\python\python311\lib\site-packages (from
scikit-learn) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.2.0 in c:\users\ezio9\appdata\local\programs\python\python311\lib\site-packages
(from scikit-learn) (3.6.0)
Requirement already satisfied: six>=1.5 in c:\users\ezio9\appdata\roaming\python\python311\site-packages (from python-
dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: colorama in c:\users\ezio9\appdata\local\programs\python\python311\lib\site-packages (from click-
>nltk) (0.4.6)
Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 23.2.1 -> 26.2.1
[notice] To update, run: C:\Users\ezio9\OneDrive\Desktop\IAU\NLP\NLP lab\.lab_work\venv\Scripts\python.exe -m pip install --
upgrade pip
('moses',)
('supposes',)
('his',)
('toeses',)
('are',)
('roses',)
('but',)
('moses',)
('supposes',)
('erroneously',)
```

Calculating probabilities in unigram models

In [2]:
```
### Calculate counts of each word

string = "moses supposes his toeses are roses but moses supposes erroneously"

unique_words = set(word_tokenize(string))

words_frq = []
for uniques in unique_words:
    frq = string.count(uniques)
    words_frq.append((uniques, frq))

words_frq
```

Out[2]:
```
[('his', 1),
 ('erroneously', 1),
 ('roses', 1),
 ('moses', 2),
 ('but', 1),
 ('supposes', 2),
 ('toeses', 1),
 ('are', 1)]
```

In [3]:
```
### Calculate probabilities:

probab = []

for count in range(len(words_frq)):
    pro = words_frq[count][1]/len(word_tokenize(string))
    probab.append((words_frq[count][0],pro))

print('The probabilities are: ', probab)
```

```
The probabilities are:  [('his', 0.1), ('erroneously', 0.1), ('roses', 0.1), ('moses', 0.2), ('but', 0.1), ('supposes', 0.2),
('toeses', 0.1), ('are', 0.1)]
```

Bigram

Bigram an n-gram consisting of two items of a sequence

- Two words that are used together to mean something specific

$$P(B \mid A) = \frac{\text{Count of } (AB)}{\text{Count of } (A)}$$

Method 1

In [4]:

```
import nltk
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

bigrams = nltk.bigrams(word_tokenize(string))
for i in bigrams:
    print(i)
```

```
('moses', 'supposes')
('supposes', 'his')
('his', 'toeses')
('toeses', 'are')
('are', 'roses')
('roses', 'but')
('but', 'moses')
('moses', 'supposes')
('supposes', 'erroneously')
```

Method 2

In [5]:

```
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

bigrams = ngrams(word_tokenize(string),2)

for i in bigrams:
    print(i)
```

```
('moses', 'supposes')
('supposes', 'his')
('his', 'toeses')
('toeses', 'are')
('are', 'roses')
('roses', 'but')
('but', 'moses')
('moses', 'supposes')
('supposes', 'erroneously')
```

Trigram

Trigram an n-gram consisting of three items of a sequence

- Three words that are used together to mean something specific

Method 1

In [6]:

```
import nltk
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

trigrams = nltk.trigrams(word_tokenize(string))

for i in trigrams:
    print(i)
```

```
('moses', 'supposes', 'his')
('supposes', 'his', 'toeses')
('his', 'toeses', 'are')
('toeses', 'are', 'roses')
('are', 'roses', 'but')
('roses', 'but', 'moses')
('but', 'moses', 'supposes')
('moses', 'supposes', 'erroneously')
```

Method 2

In [7]:

```
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

trigrams = ngrams(word_tokenize(string),3)

for i in trigrams:
    print(i)
```

```
('moses', 'supposes', 'his')
('supposes', 'his', 'toeses')
('his', 'toeses', 'are')
('toeses', 'are', 'roses')
('are', 'roses', 'but')
('roses', 'but', 'moses')
('but', 'moses', 'supposes')
('moses', 'supposes', 'erroneously')
```

What is padding in NLP?

Padding is a technique used to ensure that all input sequences have the same length. This is necessary because neural networks require inputs that have the same shape and size. However, when we pre-process and use texts as inputs for our model, not all sentences have the same length. In other words, some sentences are longer or shorter than others. We need to have inputs with the same size, and this is where padding comes in.

Padding is a special form of masking where the masked steps are at the start or the end of a sequence. Padding comes from the need to encode sequence data into contiguous batches: in order to make all sequences in a batch fit a given standard length. The padding is added before splitting the sentence.

n-1 padding is added.

In [8]:

```
from nltk.util import ngrams
from nltk.tokenize import word_tokenize


string = "moses supposes his toeses are roses but moses supposes erroneously"

print('------------------- Padding in Bigrams ------------------------------')
list(ngrams(word_tokenize(string), pad_left=True, left_pad_symbol="<s>",
                                pad_right=True, right_pad_symbol="</s>", n=2))
```

```
------------------- Padding in Bigrams ------------------------------
```

Out[8]:

```
[('<s>', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'his'),
 ('his', 'toeses'),
 ('toeses', 'are'),
 ('are', 'roses'),
 ('roses', 'but'),
 ('but', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'erroneously'),
 ('erroneously', '</s>')]
```

In [9]:

```
print('------------------- Padding in Trigrams ------------------------------')
list(ngrams(word_tokenize(string), pad_left=True, left_pad_symbol="<s>",
                                pad_right=True, right_pad_symbol="</s>", n=3))
```

```
------------------- Padding in Trigrams ------------------------------
```

Out[9]:

```
[('<s>', '<s>', 'moses'),
 ('<s>', 'moses', 'supposes'),
 ('moses', 'supposes', 'his'),
 ('supposes', 'his', 'toeses'),
 ('his', 'toeses', 'are'),
 ('toeses', 'are', 'roses'),
 ('are', 'roses', 'but'),
 ('roses', 'but', 'moses'),
 ('but', 'moses', 'supposes'),
 ('moses', 'supposes', 'erroneously'),
 ('supposes', 'erroneously', '</s>'),
 ('erroneously', '</s>', '</s>')]
```

Alternatively, We can get rid of the parameters by using

> pad_both_ends method in NLTK

Note the n argument, that tells the function we need padding for bigrams.

In [10]:

```
from nltk.lm.preprocessing import pad_both_ends
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

list(ngrams(pad_both_ends(word_tokenize(string), n=2), n=2))
```

Out[10]:

```
[('<s>', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'his'),
 ('his', 'toeses'),
 ('toeses', 'are'),
 ('are', 'roses'),
 ('roses', 'but'),
 ('but', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'erroneously'),
 ('erroneously', '</s>')]
```

Training the Maximum Likelihood Model

Maximum likelihood estimation estimates the model parameters such that the probability is maximized.

- Get the requisite n_grams frequency counts from a corpus.
- Normalize them to a 0-1 range.
- Build the vocabulary (of unique words): to create this vocabulary we need to pad our sentences (just like for counting ngrams) and then combine the sentences into one flat stream of words.

To do the above steps, we need to create an Ngram and add the padding then flatten the list. All these steps are requested for pre-processing to train the MLE model. However, we can use padded_everygram_pipeline() in NLTK to do all the steps in one call.

The padded_everygram_pipeline() will return:
1- **ngrams_gen:** Each sentence becomes a list of n-grams with padding.
2- **vocab_gen:** A flat stream of all tokens, with start and end symbols included

References:

https://www.nltk.org/api/nltk.lm.html

https://www.kaggle.com/code/alvations/n-gram-language-model-with-nltk

In [11]:

```
from nltk.lm.preprocessing import padded_everygram_pipeline
from nltk.util import ngrams
from nltk.tokenize import word_tokenize
from nltk.lm import smoothing

string = "moses supposes his toeses are roses but moses supposes erroneously"

#Create Bigram Language Model
train, vocab = padded_everygram_pipeline(2, [word_tokenize(string)])
```

In [12]:

```
from nltk.lm import MLE
lm = MLE(2)
lm.fit(train, vocab) #Train the Maximum Likelihood Model
```

Calculating Probabilities

Here's how you get the score for a word given some preceding context. For example we want to know what is the chance that "b" is preceded by "a".

In [13]:

```
print(lm.score('supposes', ['moses'])) #Calculate Bigram Probabilities for P('moses'|'supposes')
print(lm.score('moses', ['supposes'])) #Calculate Bigram Probabilities for P('supposes'|'moses') - Returns 0 because unseen
```

```
1.0
0.0
```

Evaluating n_grams models using preplexity

Preplexity is a measurement of how well a probability model (like a language model) predicts a sample.

It's the probability of the test set normalized by the number of words.

A `lower` perplexity means:

- The model is less “perplexed” by the data.
- The model assigns higher probabilities to the actual words in the test set.
- Therefore, better performance.

A `higher perplexity` = more uncertainty = worse model.

In [14]:

```
test = [('moses', 'supposes'), ('toeses','are')]
print(lm.perplexity(test)) #Perplexity for bigram model
```

```
1.0
```

Task: Generating tweets using n_grams

In this Task, we will build a MLE model with n-gram to generate tweets.

The agenda is:

1. load the data: https://www.kaggle.com/datasets/adizafar/large-random-tweets-from-pakistan
2. pre-processing the data:

- Remove hashtags
- Remove RT
- Remove any website
- Remove any mentions
- Remove emojis

3. build MLE model with n-gram (Bi-gram)
4. evaluate the model
5. calculate the probability of the bigram: (pakistan is)
6. calculate the preplexity of the word: (pakistan)

before we start we need to install library to handle the emojis in tweets

> pip install emoji

In [15]:

```
from pathlib import Path
import pandas as pd

data = pd.read_csv(
    Path("data") / "Random Tweets from Pakistan- Cleaned- Anonymous.csv",
    encoding="utf-8", encoding_errors="replace", low_memory=False
)
print("Number of tweets:", len(data))
print("Tweets with invalid encoding bytes:", data["full_text"].str.contains("�", na=False).sum())
data.head()
```

```
Number of tweets: 202202
Tweets with invalid encoding bytes: 6538
```

Out[15]:

	Unnamed: 0	created_at_tweet	full_text	retweet_count	favorite_count	reply_count	location
0	0	Wed Jul 03 07:20:15 +0000 2019	تیرا لیڈر میرا لیڈر نواز شریف نواز شریف کیا ❤️...	226	1097	127	Punjab, Pakistan
1	1	Fri Jul 02 02:50:53 +0000 2021	Happy birthday to my brother n boss , May you ...	54	273	9	Punjab, Pakistan
2	2	Fri Jul 02 12:12:53 +0000 2021	❤️❤️	50	131	13	Punjab, Pakistan
3	3	Tue Oct 15 22:29:16 +0000 2019	`suspicious °jikook au jimin'in yaşadığı kasab...	522	2134	86	İstanbul, Türkiye
4	4	Wed May 13 04:34:31 +0000 2020	Speaking of @SpiderMan... 😂 https://t.co/uGJ...	74	535	11	Follow the ATP Tour ➡️

In [16]:

```
import re
import emoji

def preprocess_tweet(text):
    text = re.sub(r"https?://\S+|www\.\S+", " ", text)
    text = re.sub(r"#\w+", " ", text)
    text = re.sub(r"\bRT\b", " ", text, flags=re.IGNORECASE)
    text = re.sub(r"@\w+", " ", text)
    text = emoji.replace_emoji(text, replace=" ")
    text = text.replace("\ufffd", " ")
    return " ".join(text.lower().split())

tweets = data["full_text"].dropna().map(preprocess_tweet)
tweets = tweets[tweets.str.len() > 0].drop_duplicates().reset_index(drop=True)
print("Cleaned unique tweets:", len(tweets))
tweets.head()
```

```
Cleaned unique tweets: 131386
```

Out[16]:

```
0    تیرا لیڈر میرا لیڈر نواز شریف نواز شریف کیا آپ...
1    happy birthday to my brother n boss , may you ...
2    `suspicious °jikook au jimin'in yaşadığı kasab...
3                                    speaking of ...
4                               alexa, skip to 2021
Name: full_text, dtype: str
```

In [17]:

```
from sklearn.model_selection import train_test_split
from nltk.lm import MLE
from nltk.lm.preprocessing import padded_everygram_pipeline
from nltk.tokenize import word_tokenize

train_tweets, test_tweets = train_test_split(tweets, test_size=0.2, random_state=42)
train_tokens = [word_tokenize(text) for text in train_tweets]
test_tokens = [word_tokenize(text) for text in test_tweets]
train_ngrams, vocabulary = padded_everygram_pipeline(2, train_tokens)
tweet_lm = MLE(2)
tweet_lm.fit(train_ngrams, vocabulary)
print("Training tweets:", len(train_tokens))
print("Test tweets:", len(test_tokens))
print("Vocabulary size:", len(tweet_lm.vocab))
```

```
Training tweets: 105108
Test tweets: 26278
Vocabulary size: 137987
```

In [18]:

```
import math
from collections import Counter
from nltk.util import ngrams
from nltk.lm.preprocessing import pad_both_ends

test_counts = Counter(
    bigram
    for tokens in test_tokens
    for bigram in ngrams(pad_both_ends(tokens, n=2), 2)
)
total_bigrams = sum(test_counts.values())
unseen_bigrams = 0
negative_log_probability = 0.0
for (previous, word), count in test_counts.items():
    probability = tweet_lm.score(word, [previous])
    if probability == 0:
        unseen_bigrams += count
    else:
        negative_log_probability -= count * math.log2(probability)

test_perplexity = math.inf if unseen_bigrams else 2 ** (negative_log_probability / total_bigrams)
print("Test bigrams:", total_bigrams)
print("Unseen test bigrams (zero probability under MLE):", unseen_bigrams)
print("Test bigram coverage:", f"{1 - unseen_bigrams / total_bigrams:.2%}")
print("Test perplexity:", test_perplexity)
```

```
Test bigrams: 532765
Unseen test bigrams (zero probability under MLE): 169778
Test bigram coverage: 68.13%
Test perplexity: inf
```

In [19]:

```
print("P(is | pakistan):", tweet_lm.score("is", ["pakistan"]))
print("Perplexity of pakistan (unigram, no preceding context):", tweet_lm.perplexity([("pakistan",)]))
```

```
P(is | pakistan): 0.043097455859863756
Perplexity of pakistan (unigram, no preceding context): 308.40205755595724
```

In [20]:

```
for seed in range(3):
    generated = tweet_lm.generate(30, text_seed=["<s>"], random_seed=seed)
    words = []
    for word in generated:
        if word == "</s>":
            break
        if word != "<s>":
            words.append(word)
    print(" ".join(words))
```

اگر پرائم منسٹر صاحب نے خواتین، نوجوانوں کو سزا ملنی چاہیے۔
: زرداری کی تو عوام کا مؤقف کو ان اقدامات کی رہی ہے ،'' case of electricity to their and `` step was expected contract has a different from lar…
چاروں کھلاڑی جارحانہ حربوں کا مک گئی تصویر میں لے مالک